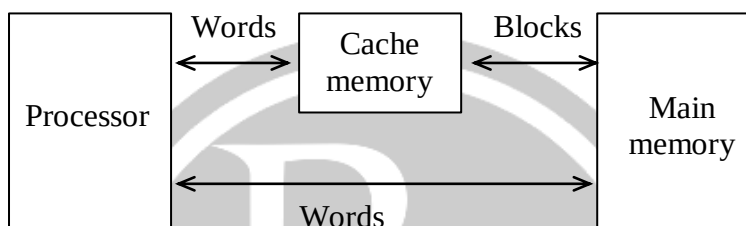


8

CACHE MEMORY

8.1. Introduction

- The speed of the main memory is very low in comparison with the speed of modern processors. For good performance an efficient solution is to use a fast cache memory.



- It is the smallest and fastest memory component in the hierarchy.
- By placing active portions of the program and data in a fast small memory, the average access time can be reduced.
- The effectiveness of cache mechanism is based on principle called “**locality of reference**”
- The Locality of reference states that, “The references to memory at any given interval of time tend to be confined within a few localized areas in memory. i.e. many instructions in localized areas of the program are executed repeatedly. It can be

1. **Temporal:** It means that recently executed instruction is likely to be executed again very soon.

2. **Spatial:** It means that instructions in close proximity to CI recently executed instruction,

Example: loops, nested loops, procedure calls. Etc.

- The performance of cache is measured using Hit ratio

$$\text{Hit ratio} = \frac{\text{no. of hits}}{\text{Total CPU references}} \times 100$$
$$\text{Hit ratio} = \frac{\text{no. of hits}}{\text{no. of hits} + \text{no. of misses}} \times 100$$

- The performance of cache can be analysed with the following characteristics.

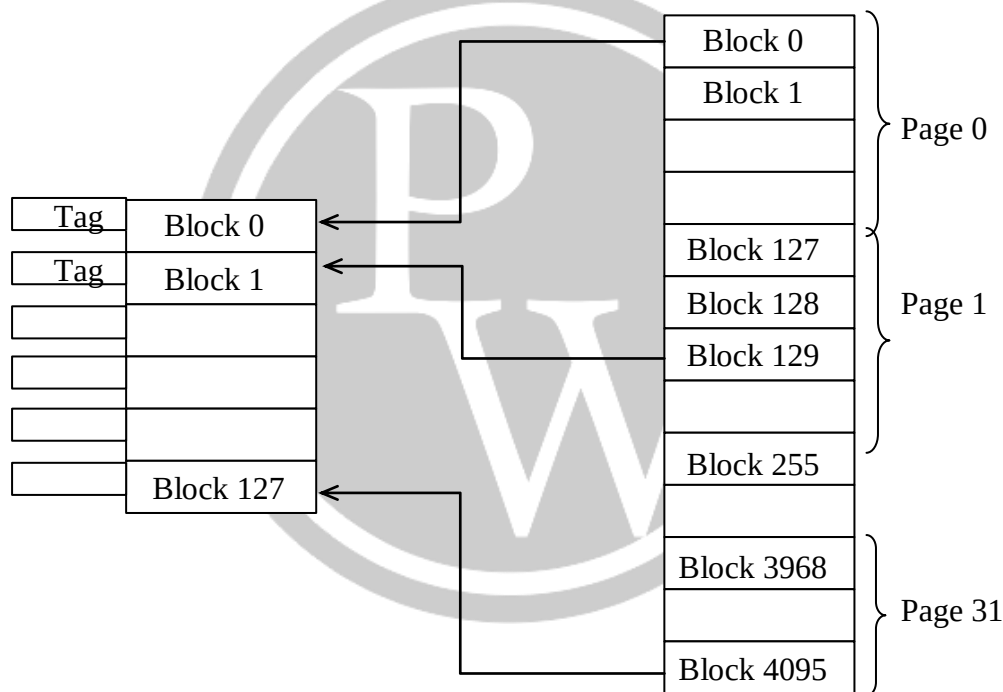
- | | |
|---------------------------------|----------------------------|
| (1) Cache size (Small in KB's) | (2) Block or line size |
| (3) No. of levels of cache | (4) Cache mapping process. |
| (5) Cache replacement algorithm | (6) Cache updating scheme. |

8.2. Cache Mapping Functions

- Consider a cache consisting of 128 blocks of 16 words cache, assume the main memory is addressable by a 16-bit address. The main memory is addressable by a 16 –bit address. The main memory has 64K words viewed as 4K blocks of 16 words each.
- The various mapping functions are
 - Direct mapping
 - Associative mapping
 - Set – Associative mapping

8.2.1 Direct Mapping

- The simplest way to determine cache locations in which to store memory blocks.
- Each block from the main memory has only one possible location in cache.
- Block j of the main memory maps to block $j \bmod n$. Where n is the no. of blocks in the cache.



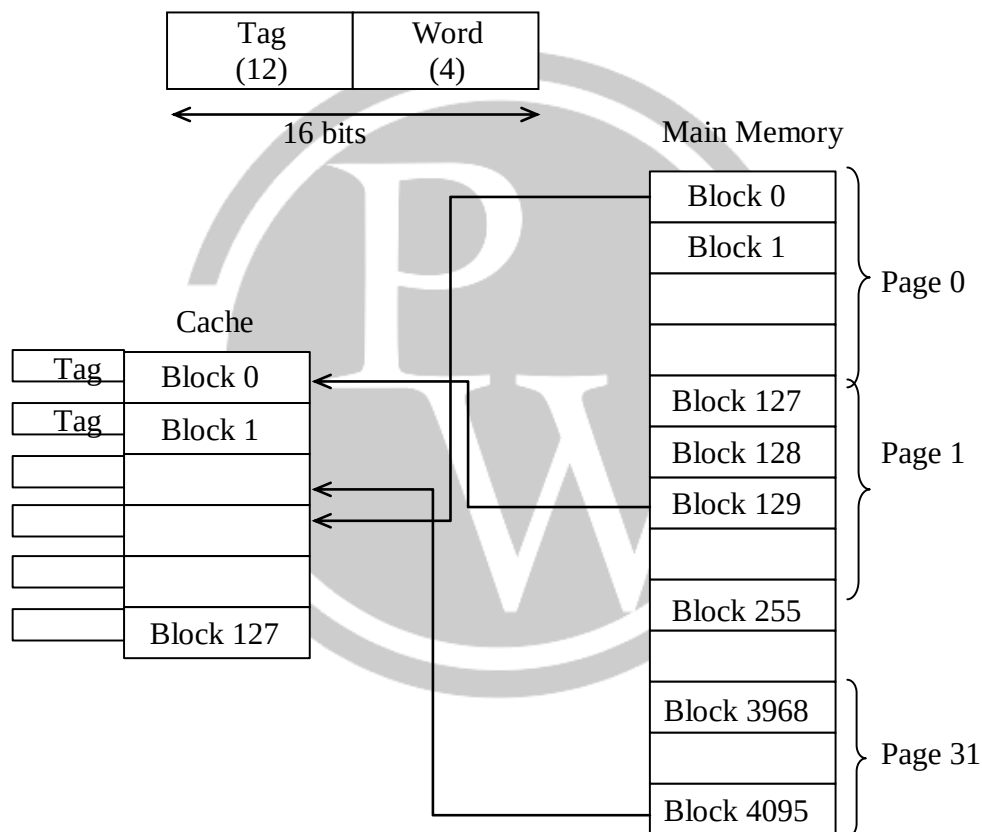
- If the processor needs to access same memory locations from two different pages of the main memory frequently, hen miss & will be more.
- Easy to implement but hit ratio is less.
- To implement this the address is divided into Three fields.
- Example :

Tag	Block	Word
5	7	4

- As execution proceeds, the higher order tag bits of the address are compared with the tag bits associated with that cache location. If they match, then the desired word is in that block of cache. If there is no match, then miss will occur.
- Requires only one comparison.

8.2.2 Associative Mapping

- In which a main memory block can be placed into any cache block position.
- It gives complete freedom in choosing the cache location in which to place the memory block. Thus the space in the cache can be used effectively.
- A new block that has to be brought into the cache has to replace an existing block if the cache is full.
- The cost is high, because of need to search all tag patterns to determine whether a given block is in the cache, i.e. comparison circuit is more complex.
- Requires maximum of n comparisons. Where n is the number of cache blocks.
- The no. of tag comparators = no. of cache blocks.
- The address is divided into 2 fields.

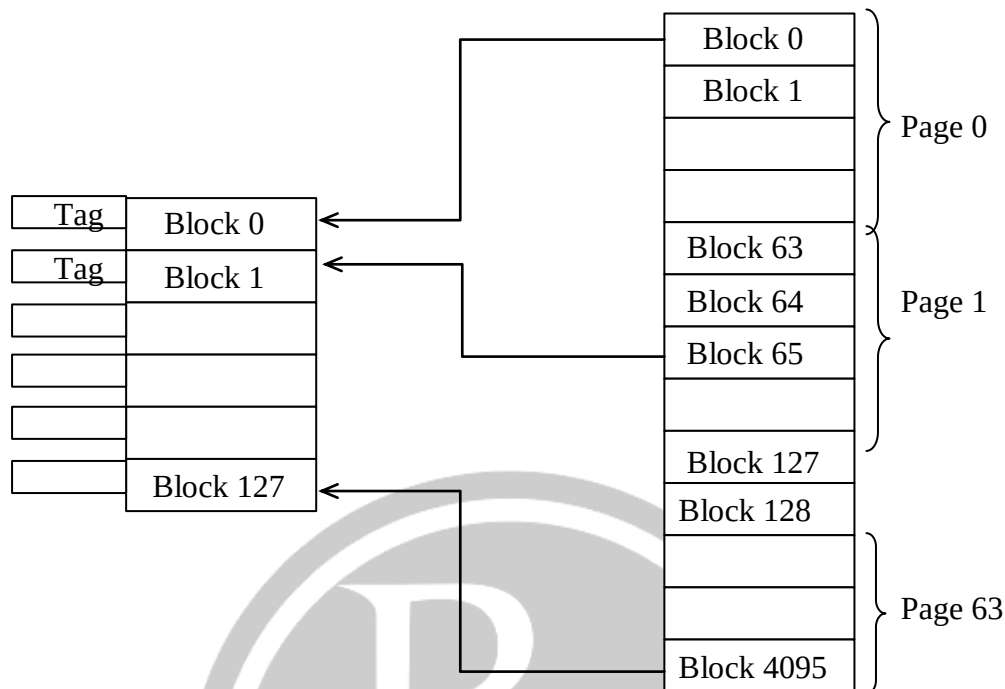


- We need an algorithm to select the block to be replaced.

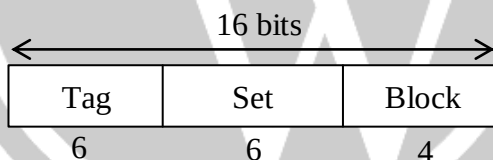
8.2.3 Set - Associative Mapping

- For efficiency, a combination of direct and associative mapping techniques can be used.
- Blocks of the cache are grouped into sets, and the mapping allows a block of the main memory to reside in any block of a specific set. i.e. contains several groups of directed mapped caches in parallel.
- The contention problem of the direct mapped method is eased by having a few choices for block placement.

- The hardware cost is reduced for comparing tags unlike associative mapping.
- A block can occupy either of blocks in the set.



- A cache that has K blocks per set, then it is referred as K-way set-associative cache.
- The address is divided into 3- fields.



- Number of tag comparisons is equal to number of blocks in the set.

8.3. Cache Replacement Policy

- In direct mapped cache, the position of each block is predetermined hence no replacement strategy exists.
- In Associative and set – Associative caches there exists some flexibility, when a new block is to be brought into the cache and all positions that it may occupy are full. Hence replacement strategy exists.
- The replacement policies are aimed for reducing the penalty (miss) to feature references.

The various block replacement policies are

- (1) **FIFO**: The block which enters first will be the candidate for replacement. i.e. Assumes that, since it has spent long time in cache all the references to it are slightly exhausted. Implemented using queue data structure.
- (2) **LRV**: The block in the set which has been in the cache longest with no references to it is selected for the replacement. (Least recently used).
- (3) **LFU**: The block in the set which has the fewest references is selected for replacement. (Least frequently used).

- (4) **Random:** No specific criteria for replacement of any block. The existing blocks are replaced randomly.

8.4. Cache Updation Policy

- The two cache updation schemes employed are
 - **Write – through:** The simplest and commonly used approach. Updation of cache and main memory are done simultaneously.
 - Main memory contains the same data as the cache. Which is important for DMA transfers.
 - **Write-back:** In this method, only the cache location is updated during a write operation. The location is then marked by a flag so that later when the word is removed from the cache it is copied into main memory.
 - i.e. updation of main memory is postponed until the cache block selected for replacement.

8.5. Cache Memory & Arrays

An array is a homogeneous collection of data items stored in contiguous memory locations either in row major order or column major order.

Example:

$$\begin{bmatrix} 3 & 1 & 2 \\ 5 & 6 & 9 \\ 8 & 4 & 7 \end{bmatrix}$$

Row major: 00 01 02 10 11 12 20 21 22

3	1	2	5	6	9	8	4	7
---	---	---	---	---	---	---	---	---

Col major: 00 10 20 01 11 21 02 12 22

3	5	8	1	6	4	2	9	7
---	---	---	---	---	---	---	---	---

Q.1. Consider an array is A[100] and each element occupies 4 – words, 32 – word cache is used and it is divided into 8 – word blocks (a) what is the hit ratio for the following instruction.

(a) For (i = 0 ; i < 100 ; i ++)
A [i] = A [i] + 10 ;

Ans.

	R	W
A[0]	M	H
A[1]	H	H
A[2]	M	H
A[3]	H	H

A(0)
A(1)
A(2)
A(3)

(b) How many times block '0' is modified in the cache memory.

Ans. 0, 8, 16, - - - - 80, 88, 96. ⇒ 13 times.

(c) How many times block '0' is replaced.

Ans. 12 times.

